An Empirical Evaluation of Quantisation Effects on Reliability and Confidence Calibration of Real-Time Edge-Based Object Detection Systems

Faizan Shafi Seroo
School of Physics, Engineering and Computer Science
University of Hertfordshire
Hatfield, United Kingdom
faizanshafi454@gmail.com

Grigorios Skaltsas
School of Physics, Engineering and Computer Science
University of Hertfordshire
Hatfield, United Kingdom
g.skaltsas@herts.ac.uk

***Abstract*—Edge-deployed object detection systems increasingly rely on post-training quantisation to meet strict latency and memory constraints on CPU-class hardware. However, most evaluations focus on accuracy and throughput, leaving the reliability of the model's confidence outputs under quantisation largely unexplored. This paper presents a three-way empirical study of confidence calibration and robustness for a YOLOv8n detector exported with OpenVINO at FP32, FP16, and INT8 precisions and deployed on an Intel Core i5 laptop. Experiments span COCO128 for controlled accuracy and Expected Calibration Error (ECE) measurement, and a real-world walking video evaluated under eight input conditions, including clean footage, three levels of Gaussian blur, three JPEG compression levels, and additive Gaussian noise. On clean video, mean confidence rises sharply with compression (0.318 for FP32, 0.571 for FP16, 0.700 for INT8), while COCO128 results show INT8 achieving the lowest ECE due to dataset-specific OpenVINO calibration. Under degraded inputs, all three precisions converge to similar mean confidence around 0.55 despite large differences in detection counts, revealing a previously unreported *confidence convergence* phenomenon. Across degradation conditions, FP16 exhibits the most stable confidence behaviour, suggesting it offers the best trade-off between throughput, calibration quality and robustness for reliability-critical edge deployments.**

***Index Terms*—quantisation, confidence calibration, reliability, YOLOv8, OpenVINO, edge computing, object detection**

I. Introduction

Edge-deployed computer vision systems are now embedded in autonomous vehicles, urban surveillance infrastructure and industrial monitoring pipelines, where object detectors must run under tight latency and memory budgets on CPU-class hardware. In this setting, post-training quantisation has become a standard technique for compressing models by reducing numerical precision from 32-bit floating point (FP32) to low-precision formats such as 16-bit floating point (FP16) and 8-bit integers (INT8), yielding substantial gains in throughput and model size without retraining [5]–[7]. While a large body of work has demonstrated that carefully calibrated INT8 pipelines can preserve mean Average Precision (mAP) within a few percentage points of FP32 baselines, evaluations overwhelmingly report only accuracy and frames-per-second metrics.

For safety-critical deployments, these metrics are incomplete. Downstream decision-making components rarely consume raw detections directly; instead, they apply thresholds and business logic to the detector's confidence scores. A model that retains mAP but becomes systematically overconfident after quantisation can be more dangerous than a slightly less accurate but honestly calibrated model, because high-confidence false positives and false negatives are more likely to pass unnoticed through monitoring and control layers. Modern deep networks are already known to be miscalibrated in their probability estimates, frequently reporting higher confidence than their empirical accuracy justifies [10], [11]. For object detection and segmentation, confidence scores are particularly difficult to calibrate because they jointly encode objectness and class probabilities, and miscalibration has been documented across a range of architectures and datasets [12].

The misalignment between evaluation practice and deployment needs is particularly acute for CPU-only edge hardware. Much of the quantisation literature targets GPU-accelerated platforms such as NVIDIA Jetson or data-centre accelerators, whereas many practical deployments rely on commodity CPUs in embedded controllers, industrial PCs or laptop-class devices [15]. Toolchains such as Intel's OpenVINO promise hardware-aware optimisations and automatic INT8 calibration on these platforms [4], [9], but their effect on the trustworthiness of confidence outputs has not been systematically studied under realistic input conditions. Recent work has begun to explore quantisation robustness to input degradations and natural distribution shifts for object detection [13], [14], yet these analyses predominantly focus on accuracy and detection counts, leaving confidence calibration and temporal stability under degradation largely unexamined.

This paper addresses these gaps through an empirical study of quantisation effects on confidence calibration and robustness for a real-time edge-based object detection system. Using the YOLOv8n architecture exported through OpenVINO at FP32, FP16 and INT8 precisions, all experiments are executed on an Intel Core i5 CPU laptop without GPU acceleration to reflect a realistic edge deployment scenario. Evaluation is

performed on two complementary data sources: the COCO128 dataset for controlled accuracy and Expected Calibration Error (ECE) measurement, and a real-world walking video processed under eight input conditions, including clean footage, three levels of Gaussian blur, three JPEG compression levels and additive Gaussian noise. Beyond standard accuracy and latency, the analysis focuses on confidence distributions, temporal stability and calibration behaviour across precision levels.

The empirical results reveal three key findings. First, on clean video, quantisation introduces a clear confidence gradient: mean confidence increases from approximately 0.32 for FP32 to 0.57 for FP16 and 0.70 for INT8, even though detection accuracy remains comparable. Second, COCO128 experiments show that INT8 achieves the lowest ECE among the three precisions, a behavior attributed to OpenVINO's dataset-specific calibration during INT8 export. Third, and most notably, under degraded input conditions all three precisions converge to a similar mean confidence around 0.55 despite large differences in detection counts, exposing a previously unreported *confidence convergence* phenomenon under input degradation. Across all conditions, FP16 exhibits the most stable confidence behaviour, suggesting it offers a favourable balance between throughput, calibration quality and robustness for reliability-critical edge deployments.

In summary, the contributions of this work are:

- A systematic three-way comparison of FP32, FP16 and INT8 confidence behaviour for YOLOv8n on CPU-only edge hardware using OpenVINO, covering both benchmark and real-world inputs.
- A joint analysis of detection accuracy, latency, confidence distributions and ECE-based calibration across multiple input degradation conditions, rather than accuracy alone.
- The identification and characterisation of a confidence convergence effect under degraded inputs, and the empirical observation that FP16 provides the most stable confidence profile across conditions, with implications for precision choice in safety-critical deployments.

II. Methodology

This section describes the experimental design used to evaluate the impact of post-training quantisation on the reliability and confidence calibration of a real-time object detection system. The pipeline is built around the YOLOv8n architecture exported through Intel's OpenVINO toolkit at three precision levels (FP32, FP16 and INT8) and deployed on CPU-only edge hardware. All experiments are conducted using publicly available models and datasets to ensure reproducibility.

A. Hardware and Software Environment

All experiments were performed on a consumer-grade laptop equipped with an Intel Core i5 CPU and 16 GB RAM, running Windows 11. No GPU acceleration was used at any stage, reflecting a realistic CPU-only edge deployment scenario. A dedicated Conda environment with Python 3.10 was created to isolate dependencies.

The core software stack comprised the Ultralytics YOLOv8 implementation for baseline FP32 inference, the OpenVINO Model Optimizer and runtime for quantised deployment [4], OpenCV for video processing and degradation, and the standard scientific Python ecosystem (NumPy, pandas, SciPy, Matplotlib) for data handling, statistics and visualisation. Library versions were pinned in a `requirements.txt` file stored alongside the code to support future replication.

B. Model Selection and Quantisation Procedure

YOLOv8n, the nano variant of the YOLOv8 family, was selected as the base detector due to its favourable trade-off between accuracy and computational cost for edge applications [3]. The network contains approximately 3.2 M parameters and achieves competitive performance on the COCO benchmark [2] while remaining lightweight enough for real-time CPU inference. The original PyTorch FP32 checkpoint was exported to OpenVINO's intermediate representation (IR) format and then converted into three precision configurations:

- **FP32**: Full-precision baseline, serving as the reference model.
- **FP16**: Half-precision floating point model, providing moderate compression with minimal numerical change.
- **INT8**: Post-training quantised model using OpenVINO's Neural Network Compression Framework.

For INT8 conversion, OpenVINO's accuracy-aware post-training quantisation was applied using a calibration subset derived from the COCO128 training images. The calibration routine estimates scale factors for activations and weights to minimise accuracy loss during 8-bit discretisation. No quantisation-aware retraining was performed; all three models share identical pre-trained weights prior to export, ensuring that any differences in behaviour arise from precision changes rather than retraining.

C. Datasets

Two complementary data sources were used:

1) COCO128: COCO128 is a 128-image subset of the COCO dataset widely used for quick benchmarking and validation [2]. It provides ground-truth bounding boxes and class labels across a diverse set of everyday scenes. In this study, COCO128 is used to compute standard detection metrics (mAP, precision, recall) as well as Expected Calibration Error, enabling controlled analysis of accuracy and calibration on clean, labelled images.

2) Real-World Walking Video: To capture deployment-like behaviour, a 360-frame video of a person walking along a city street was recorded on a consumer smartphone at 30 FPS and 1920×1080 resolution. This sequence features dynamic backgrounds, lighting variations and occlusions typical of real-world surveillance or robotics scenarios. Every frame was processed by each precision model under multiple input conditions to study confidence distributions and temporal stability in a realistic, unlabelled setting.

D. Input Degradation Pipeline

To examine robustness under realistic sensor and transmission artefacts, the walking video was processed under eight input conditions:

- Clean (no degradation).
- Gaussian blur with standard deviations $\sigma \in \{1, 2, 3\}$.
- JPEG compression with quality factors $Q \in \{90, 70, 50\}$.
- Additive Gaussian noise with zero mean and fixed variance.

Blur was implemented using OpenCV's Gaussian kernel with kernel size chosen to match the specified σ. JPEG artefacts were introduced by re-encoding frames at the target quality factor and decoding them back to pixel arrays. Noise was injected by adding Gaussian noise to each channel, clipping to valid pixel ranges. The same degradation pipeline was applied identically for each precision configuration to ensure fair comparison.

E. Evaluation Metrics

The methodology captures both conventional performance indicators and confidence-oriented metrics.

1) Accuracy and Latency: On COCO128, standard COCO-style metrics were computed: mAP@0.5 (mAP50), mAP@0.5:0.95, precision and recall. For the walking video, where ground truth is unavailable, accuracy metrics are not reported. End-to-end latency was measured as the per-frame inference time for each precision configuration, averaged over the full sequence and converted to frames per second (FPS). All measurements include pre- and post-processing overheads within the OpenVINO pipeline.

2) Confidence Distribution and Temporal Stability: For each model, all detection confidences on the walking video were recorded across frames and degradation conditions. These were summarised using:

- Mean and standard deviation of confidence scores.
- Histogram-based confidence distributions.
- Frame-wise rolling averages to assess temporal stability and jitter.

This analysis quantifies how quantisation shifts the shape and spread of confidence values, and how stable these values remain under degradation.

3) Calibration Measurement: Calibration was assessed on COCO128 using Expected Calibration Error (ECE) computed over confidence bins in the range $[0, 1]$. For each confidence bin, the empirical accuracy (fraction of correct detections) was compared with the average predicted confidence, and ECE was obtained as the weighted average of these discrepancies. Reliability diagrams were generated for each precision level to visualise over- or under-confidence across the confidence range.

To compute ECE, detections were matched to ground-truth objects using the usual Intersection-over-Union (IoU) threshold of 0.5. A detection was considered correct if its predicted class matched the ground-truth label and IoU exceeded the threshold. All unmatched detections were treated as incorrect, and all ground-truth objects without matches contributed indirectly through false negatives affecting precision and recall.

TABLE I
BASELINE PERFORMANCE ON COCO128 FOR THE THREE PRECISION LEVELS.

Precision	mAP@0.5	mAP@0.5:0.95	FPS
FP32	0.607	0.409	5.5
FP16	0.610	0.412	10.1
INT8	0.617	0.415	19.0

F. Statistical Analysis

To test the significance of observed differences between precision levels, non-parametric statistical tests were applied. For pairwise comparisons of confidence distributions and mean confidence values, the Mann–Whitney U-test was used due to non-Gaussian score distributions. Where appropriate, Kruskal–Wallis tests were used for multi-group comparisons. Effect sizes were quantified using rank-biserial correlation or equivalent measures to complement p-values.

For latency and mAP metrics, simple descriptive statistics (means and standard deviations) were reported, as the differences between precision levels were large enough that formal hypothesis tests were not strictly necessary. All statistical analysis was conducted in Python using SciPy.

III. EXPERIMENTS AND RESULTS

This section reports the empirical findings for the three precision configurations under both benchmark and deployment-like conditions. Results are organised into baseline accuracy and speed on COCO128, confidence behaviour on real-world video, robustness under input degradation and calibration quality.

A. Baseline Accuracy and Latency on COCO128

Table I summarises the COCO128 performance for the FP32, FP16 and INT8 models. All three configurations achieve comparable accuracy, with mAP differences within a narrow band, while latency differs substantially.

The INT8 model achieves roughly a $3.5\times$ throughput increase over FP32 while slightly improving mAP@0.5. FP16 provides an intermediate trade-off, almost doubling FPS relative to FP32 with negligible change in accuracy. These results confirm that post-training quantisation via OpenVINO can preserve detection performance while substantially reducing latency on CPU-only hardware.

B. Confidence Behaviour on Real-World Video

To examine how quantisation affects detector confidence in a deployment-like setting, each model was run on the clean walking video. All detection confidences were collected across 360 frames. Figure 1 shows the confidence distributions, while Figure 2 illustrates frame-wise confidence over time.

On clean input, mean confidence increases monotonically with compression: FP32 exhibits a low mean confidence of approximately 0.32, FP16 increases this to about 0.57, and

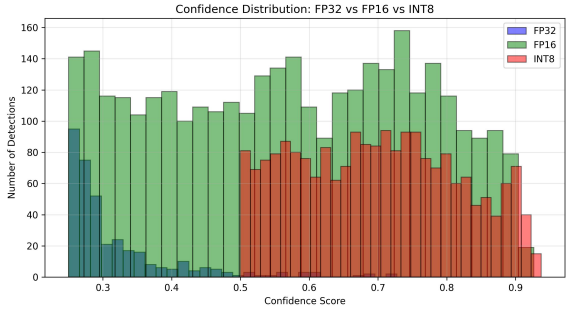

Fig. 1. Confidence distribution for FP32, FP16 and INT8 on the clean walking video.

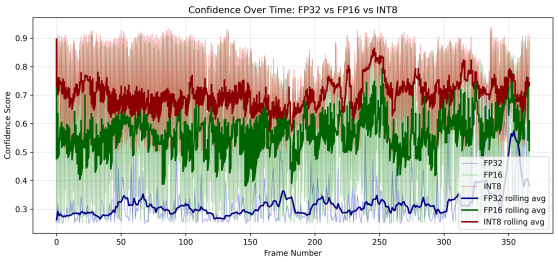

Fig. 2. Per-frame confidence over time for the three precision levels, with rolling averages overlaid.

INT8 further raises it to around 0.70. The FP32 distribution is concentrated in the 0.25–0.40 range, indicating cautious predictions, whereas INT8 produces a broad distribution skewed toward higher confidences. FP16 lies between these extremes, both in mean and spread. Temporal plots reveal that FP16 maintains smoother confidence trajectories across frames than FP32, which shows high jitter, and INT8, which exhibits abrupt jumps into very high-confidence regimes.

These results show that quantisation does not merely rescale confidences by a constant factor; it changes the entire distribution and temporal dynamics. In particular, INT8 tends to produce fewer low-confidence detections and many more high-confidence detections, which can be problematic if confidence thresholds are tuned assuming FP32 behaviour.

C. Robustness Under Input Degradation

The degradation experiments evaluate how confidence behaviour and throughput change when the walking video is corrupted by blur, compression and noise. Table II summarises the mean confidence, confidence standard deviation and FPS for each precision and degradation condition.

Two patterns emerge. First, degradation conditions dramatically increase the number of detections relative to the clean video, particularly for FP32 and FP16, as the models produce multiple overlapping boxes in response to artefacts. Second, the large confidence gap observed on clean input collapses under degradation: across blur, JPEG and noise, all three precisions converge to a mean confidence close to 0.55. This confidence convergence effect indicates that degradation

Fig. 3. Example frame from the walking video under all degradation conditions: three levels of Gaussian blur, three JPEG compression levels and additive Gaussian noise.

TABLE II
SUMMARY STATISTICS ACROSS DEGRADATION CONDITIONS FOR EACH PRECISION LEVEL.

Model	Condition	Detections	Mean conf.	FPS
FP32	Clean	367	0.318	5.5
FP32	Blur $\sigma=1$	3284	0.566	9.0
FP32	Blur $\sigma=2$	2948	0.555	6.8
FP32	Blur $\sigma=3$	2342	0.536	8.0
FP32	JPEG Q=90	3379	0.565	8.7
FP32	JPEG Q=70	3339	0.564	8.7
FP32	JPEG Q=50	3248	0.558	8.6
FP32	Gaussian noise	2792	0.514	9.3
FP16	Clean	3404	0.571	10.1
FP16	Blur $\sigma=1$	3294	0.569	24.7
FP16	Blur $\sigma=2$	2940	0.557	25.0
FP16	Blur $\sigma=3$	2317	0.536	26.4
FP16	JPEG Q=90	3377	0.570	26.3
FP16	JPEG Q=70	3356	0.566	25.4
FP16	JPEG Q=50	3293	0.558	25.5
FP16	Gaussian noise	2755	0.518	26.2
INT8	Clean	2121	0.700	19.0
INT8	Blur $\sigma=1$	3269	0.574	28.6
INT8	Blur $\sigma=2$	2918	0.564	29.3
INT8	Blur $\sigma=3$	2276	0.547	28.8
INT8	JPEG Q=90	3351	0.576	29.3
INT8	JPEG Q=70	3316	0.573	28.9
INT8	JPEG Q=50	3261	0.566	29.7
INT8	Gaussian noise	2767	0.526	28.9

erodes the overconfidence introduced by quantisation on clean data, even though detection counts and spatial accuracy still differ between precisions.

Across all conditions, FP16 maintains a stable mean confidence and exhibits the smallest variation in FPS, suggesting it provides the most predictable behaviour under changing image quality. FP32 shows lower confidence and lower throughput, while INT8 remains the fastest but also the most aggressive in assigning high confidence on clean frames.

D. Calibration Analysis

Calibration quality was evaluated on COCO128 using ECE and reliability diagrams. Figure 4 presents the reliability curves for the three precision configurations, with the diagonal representing perfect calibration.

Despite its aggressive compression, the INT8 model achieves the lowest ECE (approximately 0.17), marginally outperforming FP32 and FP16. The reliability curves show that FP32 and FP16 are mildly overconfident across most confidence bins, with empirical accuracy lying below the diagonal, whereas INT8 lies closer to the ideal line over a broad range. This behaviour is consistent with the use of COCO128 images as the calibration set during INT8 export: OpenVINO's calibration routine explicitly optimises the mapping between

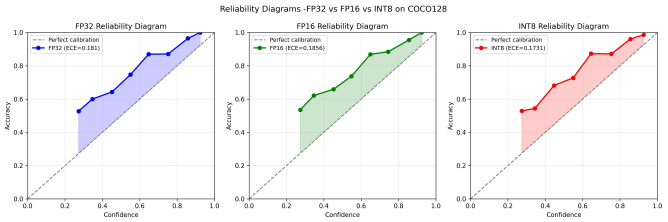

Fig. 4. Reliability diagrams for FP32, FP16 and INT8 on COCO128, with ECE values indicated in the legends.

activations and quantised scales on this dataset. However, the degradation experiments demonstrate that this calibration advantage does not fully carry over to degraded real-world inputs, where confidence convergence occurs regardless of precision.

Overall, the results indicate that quantisation affects not only accuracy and latency, but also the calibration and robustness of detector confidence. FP16 emerges as a strong default choice for CPU-only edge deployments, offering a balance between speed, stability and calibration, while INT8 requires more careful threshold tuning and validation under expected deployment conditions.

IV. Discussion and Conclusion

The results show that post-training quantisation affects far more than accuracy and latency for edge-based object detection. Although INT8 achieves the highest throughput and slightly higher mAP@0.5 than FP32 on COCO128, its confidence distributions and temporal behaviour differ substantially from the full-precision baseline. On clean video, INT8 consistently assigns higher confidence to detections than FP32 and FP16, increasing the risk that misdetections cross fixed decision thresholds if those thresholds were tuned assuming FP32 behaviour.

The degradation experiments highlight that input quality strongly mediates these quantisation effects. Under blur, compression and noise, the large confidence gap between precision levels collapses and all three models converge to a similar mean confidence around 0.55. This confidence convergence effect suggests that, in realistic deployment conditions where inputs are rarely pristine, the apparent overconfidence of INT8 on clean data may be partially neutralised by corruption-induced uncertainty. However, the simultaneous increase in detection counts under degradation indicates that detectors may respond to artefacts by emitting many overlapping boxes, which can burden downstream systems even when average confidence falls.

Calibration analysis on COCO128 reveals that INT8 can in fact be slightly better calibrated than FP32 and FP16 when the calibration dataset used during export matches the evaluation distribution. This confirms that OpenVINO's calibration procedure can correct some of the miscalibration introduced by quantisation. At the same time, the convergence of confidence under degraded real-world inputs indicates that calibration measured on clean benchmark images does not fully predict behaviour under deployment conditions, reinforcing the need to validate confidence outputs on representative corrupted data.

Taken together, the findings support FP16 as a strong default choice for CPU-only edge deployments: it offers a substantial speed-up over FP32, displays more stable confidence trajectories than INT8 and maintains competitive calibration performance. INT8 remains attractive when maximum throughput is required, but its use should be accompanied by careful threshold tuning and validation under the full range of expected input conditions. Future work will extend this study to larger datasets, additional object detection architectures and GPU-based deployment, as well as explore post-hoc recalibration techniques tailored to quantised detectors under distribution shift.

Acknowledgment

The author thanks Dr. Grigorios Skaltsas for his supervision and guidance throughout this project.

References

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.

[2] T.-Y. Lin, M. Maire, S. Belongie *et al.*, "Microsoft COCO: Common objects in context," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2014, pp. 740–755.

[3] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," GitHub repository, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[4] Intel Corporation, "OpenVINO toolkit: Open source toolkit for optimizing and deploying AI inference," GitHub repository, 2023. [Online]. Available: https://github.com/openvinotoolkit/openvino

[5] J. Ahn, J. Kim, and J. Kim, "Performance characterization of using quantization for DNN inference on edge devices," *arXiv preprint* arXiv:2303.05016, 2023.

[6] R. Boddu and A. Mukherjee, "Efficient edge deployment of quantized YOLOv4-Tiny for aerial emergency object detection on Raspberry Pi 5," *arXiv preprint* arXiv:2506.09300, 2025.

[7] Z. Yuan, J. Liu, B. Wu *et al.*, "Benchmarking the reliability of post-training quantization: A particular focus on worst-case performance," *arXiv preprint* arXiv:2303.13003, 2023.

[8] M. Williams and N. Aletras, "On the impact of calibration data in post-training quantization and pruning," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024, pp. 10100–10118.

[9] C. G. Raju, "Optimizing YOLOv8: OpenVINO standard quantization vs accuracy-controlled for edge deployment," *Indonesian J. Elect. Eng. Comput. Sci.*, vol. 37, no. 1, pp. 45–52, 2025.

[10] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 1321–1330.

[11] M. Minderer, J. Djolonga, R. Romijnders *et al.*, "Revisiting the calibration of modern neural networks," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 34, 2021, pp. 15682–15694.

[12] F. Küppers, A. Haselhoff, J. Kronenberger, and J. Schneider, "Confidence calibration for object detection and segmentation," in T. Fingerscheidt, H. Gottschalk, and S. Houben, Eds., *Deep Neural Networks and Data for Automated Driving*. Cham, Switzerland: Springer, 2022, pp. 225–250.

[13] A. Karimov, O. Tursunov, and I. Sobirov, "Quantization robustness to input degradations for object detection," *arXiv preprint* arXiv:2508.19600, 2025.

[14] X. Mao, Y. Tian, Y. Ye, and C. Shen, "COCO-O: A benchmark for object detectors under natural distribution shifts," *arXiv preprint* arXiv:2307.12730, 2023.

[15] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, 2016.